
Computação Paralela e Distribuída (CPD)
Turma T07 • Grupo 13 • 2025/2026

Concurrent Multi-User Chat Server

Design and Implementation of a Room-Based Chat System

André Pinho	up202307008
Duarte Martins	up202304549
Maria Beatriz Leite	up202307229

Contents

1	Introduction	3
2	Development Environment	3
3	System Architecture	4
3.1	Codebase Structure	5
3.2	Protocol Design	5
4	Connection State Machine	6
5	Bounded Blocking Queue	7
5.1	Lock Granularity Analysis	8
6	Room System Design	9
6.1	AI Integration and Commands	10
7	Session Recovery	11
8	Bonus Implemented Features	12
9	Testing and Validation	12
10	AI Usage	13
11	Conclusion	13
12	GitLab Repository and Work Distribution	13

List of Figures

1	Overall system architecture showing the server, handlers, queues, rooms, and connected users.	4
2	Connection lifecycle state machine for each connected client.	6
3	Bounded Blocking Queue (this example has capacity 8, count 5) showing the head, tail, and synchronisation primitives.	7
4	Room system design showing public and private room components and their relationships as well as their ability to store and share their message history.	9
5	Sequence diagram for client session recovery after a connection loss.	11

1 Introduction

The goal of this project was to design and implement a concurrent, multi-user chat server in Java, building on the concurrency primitives and parallel thinking developed in the first assignment. The system allows multiple users to connect simultaneously, authenticate, create or join chat rooms, exchange messages in real time, and recover their session automatically if the connection drops unexpectedly.

The central challenge was managing shared mutable state — connected users, active rooms, message queues, and session tokens — without introducing race conditions or deadlocks. Rather than relying on high-level concurrent collections for everything, we implemented key synchronisation structures.

The system is structured around three main concerns:

1. **Concurrency** — Each client is handled by a dedicated thread. Messages between threads flow through bounded blocking queues.
2. **Protocol correctness** — The connection lifecycle is modelled as an explicit state machine, ensuring clients cannot perform invalid actions (e.g. sending messages before authenticating).
3. **Resilience** — Sessions are tracked by a token-based manager, allowing users to reconnect and resume exactly where they left off after a network disruption. This, coupled with the ability to store message logs from each room, ensures the user that reconnecting does not mean losing the chat they were previously having.

Additionally, the server includes an AI-powered room backed by a locally running `gemma3:1b` language model, accessible via HTTP, giving users the ability to chat with an AI assistant within the same interface.

2 Development Environment

The official release was tested on our personal laptops // ANDRE COMPLETA

3 System Architecture

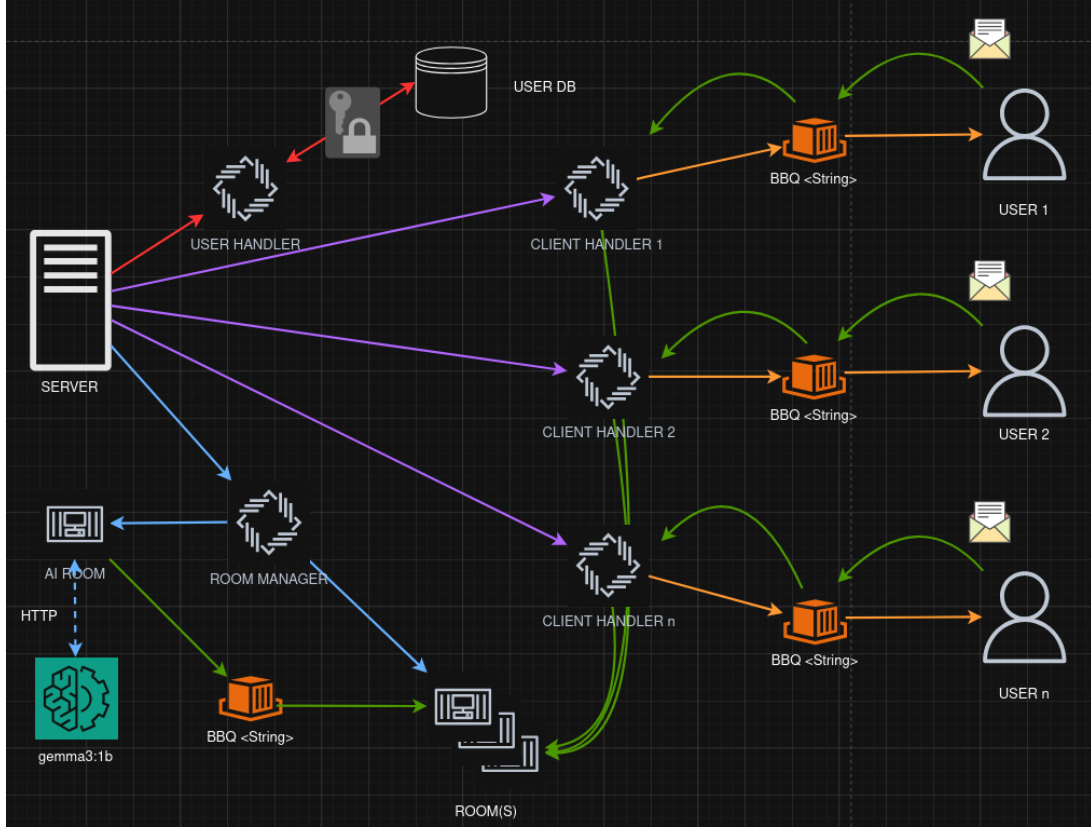


Figure 1: Overall system architecture showing the server, handlers, queues, rooms, and connected users.

The system follows a thread-per-client model (Figure 1). When a new TCP connection arrives, the main server loop spawns a dedicated **ClientHandler** thread for that connection. This handler is responsible for reading messages from the client, processing them according to the current protocol state, and dispatching responses.

Outgoing messages to a user are not written directly on the handler thread. Instead, each client has an associated **BBQ<String>** — a bounded blocking queue — that acts as a staging buffer. A separate writer thread drains this queue and sends messages over the socket. This separation means a slow or stalled client does not block the server from processing other events: the handler drops the message into the queue and moves on immediately.

User authentication data (usernames and hashed passwords) is stored in a persistent **USER DB** file, accessed through the **UserHandler** component, which holds the single lock on that resource.

Room management is centralised in the **RoomManager**. Rooms maintain their own membership sets, optionally store a password for private access, and append messages to a persistent history file. The **RoomManager** also owns a shared **BBQ<String>** for broadcasting room-level messages to all current members.

A special **AI Room** is also managed by the **RoomManager**. It communicates with a locally running **gemma3:1b** language model via HTTP, forwarding user messages to the model and relaying its responses back into the room.

We designed this system with one thread per client because it keeps the code genuinely easy to reason about — each handler is a straight-line loop that reads a message, figures out what to do, and responds. Other alternatives, like an event-driven non-blocking approach, would scale better under very heavy load but would have been significantly harder to implement and debug correctly in the time available. For the expected concurrency of this project, we figured that the thread-per-client model is more than sufficient, and the blocking queue between handler and writer already removes the main scalability concern (a slow client stalling the server). Centralising room management in a single `RoomManager` also made it straightforward to maintain consistent room state without needing distributed locking across multiple components.

3.1 Codebase Structure

The source code is organized into files with specific responsibilities:

- **Server components:** `Server.java` accepts TCP connections and dispatches threads, while `ClientHandler.java` manages the per-client state machine.
- **Client components:** `Client.java` provides the terminal UI and handles reconnection, utilizing `Ansi.java` for color output.
- **Core managers:** `RoomManager.java`, `SessionManager.java`, and `UserManager.java` handle rooms, tokens, and credentials.
- **Data structures and protocol:** `BoundedBlockingQueue.java` provides thread-safe buffering, and `Protocol.java` defines the binary framing layer.
- **Room entities:** `Room.java` handles participants and history, extended by `AIRoom.java` for Ollama-backed AI interactions.

3.2 Protocol Design

All communication between client and server uses a custom binary framing protocol. Each message consists of a 4-byte length prefix, a 1-byte type opcode, and a variable-length UTF-8 payload. Supported opcodes include:

- `MSG` and `BROADCAST` for chat payloads.
- `AUTH`, `AUTH_OK`, and `REGISTER` for handshake and authentication.
- `JOIN`, `LEAVE`, `LIST_ROOMS`, and `ROOM_LIST` for room management.
- `RECONNECT` for session recovery.
- `LOGOUT`, `LOGOUT_OK`, and `ERROR` for termination and errors.

4 Connection State Machine

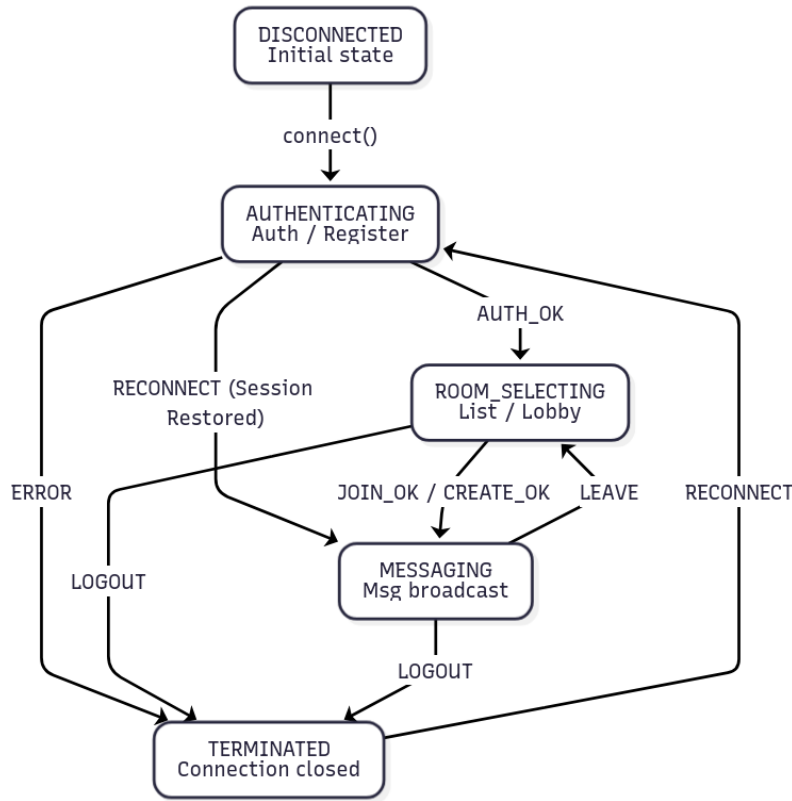


Figure 2: Connection lifecycle state machine for each connected client.

Each `ClientHandler` tracks an explicit connection state (Figure 2). A new connection begins in `DISCONNECTED` and immediately transitions to `AUTHENTICATING` when `connect()` is called. From there:

- A successful login or registration (`AUTH_OK`) moves the client to `ROOM_SELECTING`, where it can list available rooms, create a new one, or join an existing one.
- Joining or creating a room (`JOIN_OK / CREATE_OK`) transitions the client to `MESSAGING`, where message broadcast is active.
- Leaving a room (`LEAVE`) returns the client to `ROOM_SELECTING`.
- Reconnecting with a valid session token (`RECONNECT`) can restore the client directly to `ROOM_SELECTING`, or back into `MESSAGING` if the session includes an active room.
- `LOGOUT` or an unrecoverable `ERROR` transitions to `TERMINATED`, closing the connection cleanly.

We modelled the connection lifecycle as an explicit state machine rather than relying on implicit checks scattered across the handler code. The reason was practicality: early prototypes without a formal state machine produced subtle bugs where clients could, for instance, broadcast messages before completing authentication, or attempt to join a room

while already in one. Making the state explicit and only accepting certain messages in certain states means invalid transitions are caught at a single point and either ignored or returned as an error, which is much cleaner than populating the codebase with `if (authenticated)` guards everywhere.

5 Bounded Blocking Queue

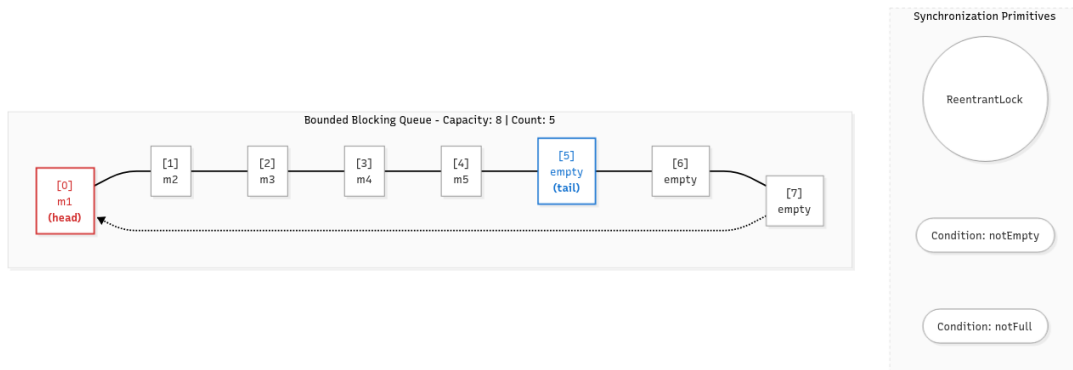


Figure 3: Bounded Blocking Queue (this example has capacity 8, count 5) showing the head, tail, and synchronisation primitives.

The `BBQ<String>` is a fixed-capacity, thread-safe circular queue (Figure 3). It is backed by an array of N slots; a `head` pointer tracks the next element to be consumed, and a `tail` pointer tracks the next free slot for insertion. When the queue is full, a producing thread blocks on the `notFull` condition; when it is empty, a consuming thread blocks on the `notEmpty` condition. Both conditions share a single `ReentrantLock`, which serialises access to the array and to the head and tail indices.

In Figure 3, the queue has a capacity of 8 and currently holds 5 messages (m1 through m5). The head points to slot 0 (the next message to be delivered) and the tail points to slot 5 (the next free slot). Slots 5, 6, and 7 are empty and available for new messages.

The BBQ is used in two places in the system:

- **Per-client outgoing queue** — Each `ClientHandler` has one `BBQ<String>` through which the handler enqueues messages destined for that client. A dedicated writer thread dequeues and sends them over the socket.
- **Room broadcast queue** — The `RoomManager` maintains a shared `BBQ<String>` for room-wide messages. When a user sends a message, it is placed here and the room’s broadcast thread fans it out to all member queues.

We implemented our own BBQ rather than using Java’s built-in `ArrayBlockingQueue` primarily because the assignment asked us to demonstrate the use of low-level synchronisation primitives, and partly because rolling our own gave us full visibility into the queue’s state for debugging. Using a `ReentrantLock` with two `Condition` objects follows the classic textbook pattern and keeps the synchronisation logic explicit and auditable. The circular array is the natural backing structure: it avoids dynamic allocation, keeps all operations $O(1)$, and is straightforward to reason about. The bounded capacity is also

deliberate — an unbounded queue would hide backpressure problems (a fast producer flooding a slow consumer) rather than surfacing them early, which is exactly the kind of issue we want to be visible.

5.1 Lock Granularity Analysis

Beyond the BBQ, the system employs a multi-granularity locking strategy tailored to each shared resource:

- **UserManager:** Utilizes a `ReentrantReadWriteLock` globally, optimizing for a read-heavy workload where authentications vastly outnumber registrations.
- **SessionManager and RoomManager:** Employ global `ReentrantLock` instances to protect shared structures from concurrent modifications.
- **Room State:** Each room uses a fine-grained `ReentrantLock` for independent protection of participants and history, reducing contention.
- **Queues and Handlers:** Each `BoundedBlockingQueue` and `ClientHandler` list uses a `ReentrantLock` to isolate independent components and protect duplicate user detection.

6 Room System Design

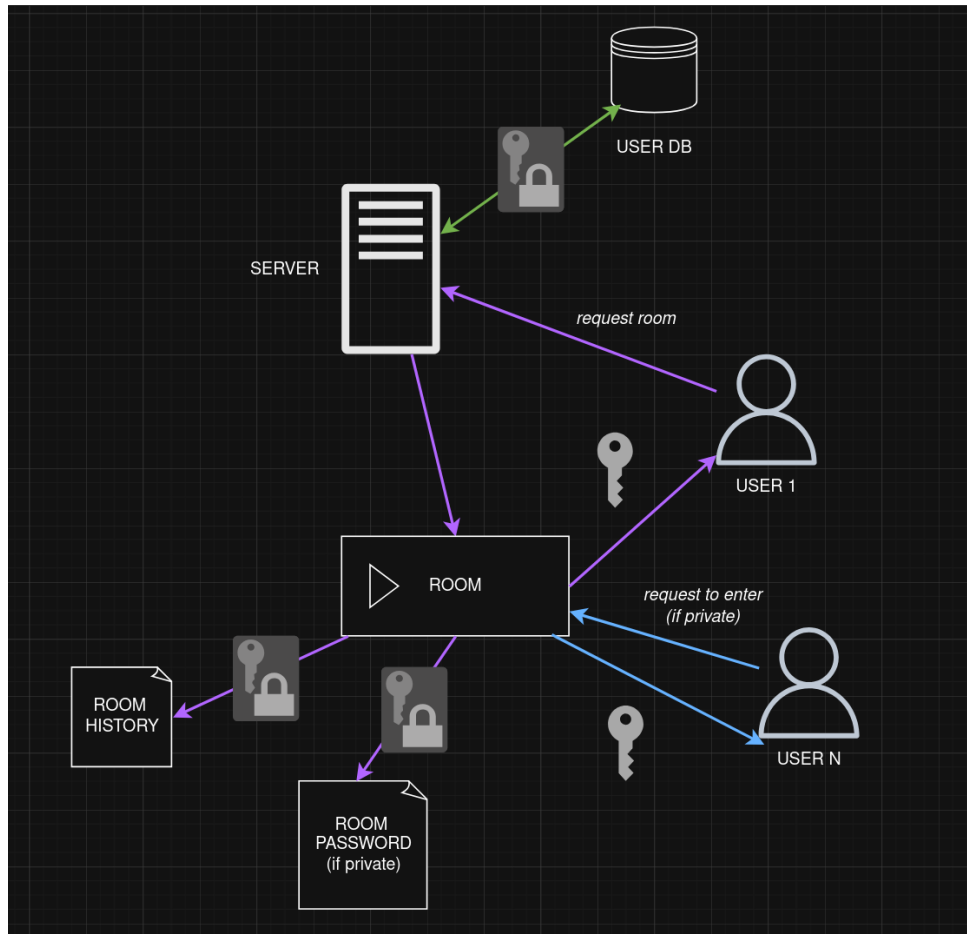


Figure 4: Room system design showing public and private room components and their relationships as well as their ability to store and share their message history.

Rooms are the central shared resource of the chat system (Figure 4). Each room holds a set of currently connected users and maintains two optional persistent files: a **room history** file that logs all messages ever sent in the room, and a **room password** file used for access control in private rooms. Access to both files is protected by a per-room lock.

When a user joins a room, the history is read back and forwarded to them so they can catch up on prior messages. When a user sends a message, it is appended to the history file synchronously before being broadcast to the current members.

Private rooms require users to supply the correct password at join time. The password is stored hashed on disk. If an incorrect password is provided, the server responds with an error and the user remains in **ROOM_SELECTING** state (Section 4).

The server maintains a registry of all active rooms through the **RoomManager**. When the last user leaves a room, the room remains registered and its history is preserved, but it is marked as empty. This means a room is never lost just because it temporarily has no members.

Storing room history in a file rather than in memory was a deliberate choice: it means message history survives server restarts without needing a separate database, and

it keeps the room’s memory footprint small even for long-lived rooms with large histories. We considered keeping history in an in-memory list and flushing it to disk asynchronously, but that introduced a more complex crash-recovery concern that was not worth the added complexity. Synchronous writes are simpler and give us strong durability guarantees essentially for free. The trade-off is disk I/O on every message, but for a chat application at this scale that is entirely acceptable.

6.1 AI Integration and Commands

The `AIRoom` extension supports commands prefixed with `/`, which the `ClientHandler` intercepts and processes. These include `/ask` for direct questions, `/summarize` for context summaries, and `/translate` for language translation. The system connects to the local Ollama instance via HTTP POST requests to the `/api/generate` endpoint. It utilizes manual JSON parsing via an `extractJsonField()` method, which searches for quoted keys and unescapes values, eliminating the need for external JSON libraries.

7 Session Recovery

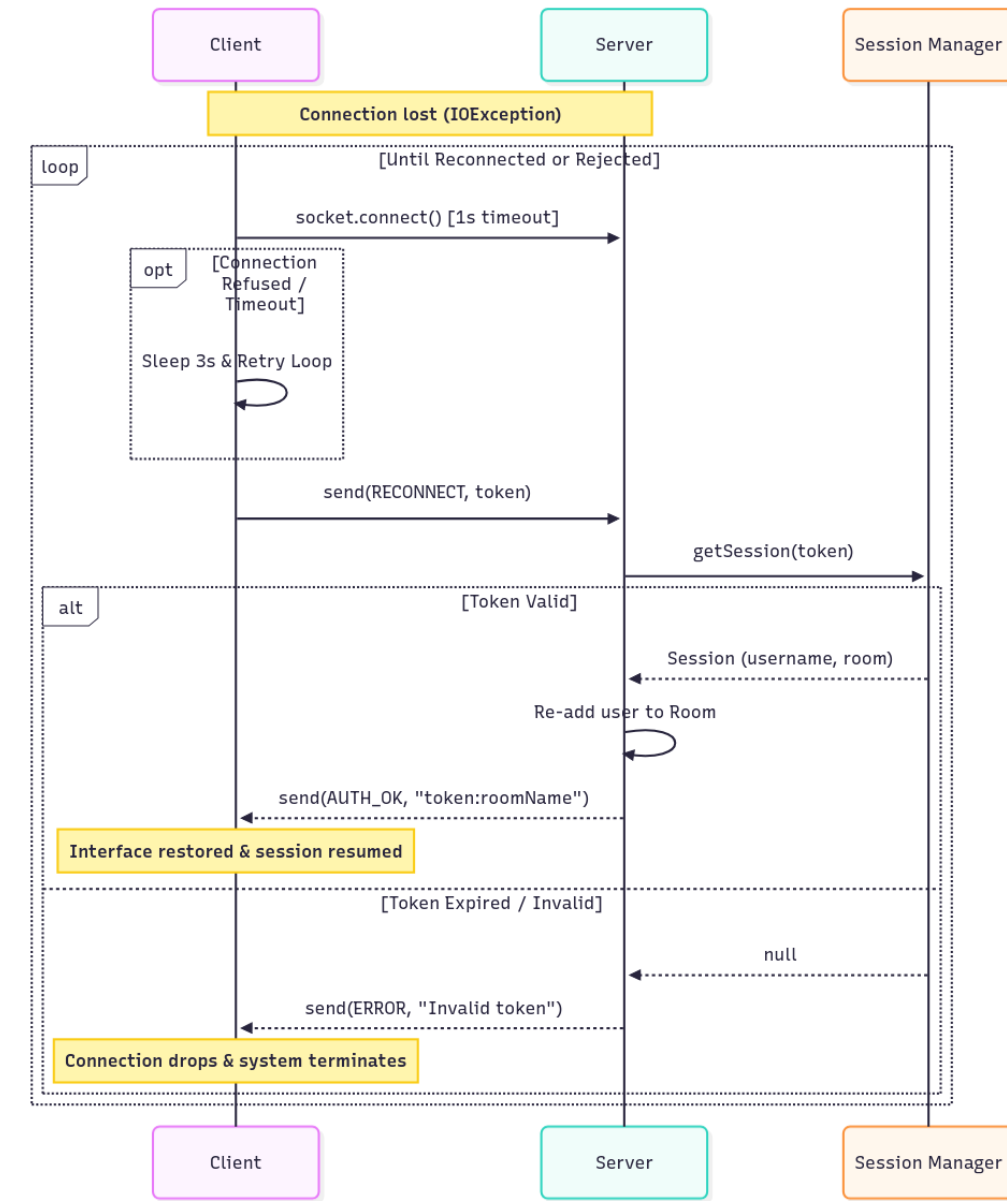


Figure 5: Sequence diagram for client session recovery after a connection loss.

When a client loses its connection (an `IOException` on the socket), it enters an automatic reconnection loop (Figure 5). The loop calls `socket.connect()` with a 1-second timeout; if the connection is refused or times out, it sleeps for 3 seconds and retries. This continues until a connection is re-established or the client gives up.

Once the TCP connection is restored, the client sends a `RECONNECT` message carrying its session token. The server passes this token to the `SessionManager`, which looks up the associated session. Two outcomes are possible:

- **Token valid** — The `SessionManager` returns the stored session (username and room name). The server re-adds the user to their previous room, restores the client interface, and sends back an `AUTH_OK` carrying the token and room name. The session resumes seamlessly.

- **Token expired or invalid** — The `SessionManager` returns `null`. The server sends an `ERROR` with “Invalid token” and drops the connection. The client must log in again from scratch.

We went with token-based session recovery because forcing users to log in again after every brief network hiccup felt like poor user experience for a real-time chat application. The token is issued on successful authentication and stored server-side in the `SessionManager`, which maps tokens to session objects containing the username and last active room. The client keeps its token in memory and sends it on reconnect. One deliberate choice was to keep token validation server-side rather than making tokens self-verifying: this makes revocation trivial — the session manager simply discards the entry — and keeps the implementation straightforward without needing cryptographic signing infrastructure. The retry loop with a fixed 3-second sleep is simple and effective; a fancier strategy with jitter and exponential backoff would be better for production use, but does not meaningfully change the outcome in the scope of this project.

8 Bonus Implemented Features

Beyond core requirements, several quality-of-life enhancements were added:

- **Custom User Commands:** The client interface supports slash commands such as `/help`, `/list`, `/join`, `/leave`, and `/logout`. Upon executing `/leave`, the server automatically returns the room list.
- **UI Enhancements:** Command feedback and tags (e.g., `[Private]`, `[AI]`) use ANSI color codes for visual clarity.
- **TLS/SSL Support:** Both the server and client support optional TLS encryption enabled via the `ssl` system property. This wraps connections using `SSLServerSocket` and `SSLSocketFactory` backed by a Java `KeyStore`.

9 Testing and Validation

The project ensures correctness under concurrent workloads through automated tests executed via Gradle:

- **Unit Tests:** Scenarios include `RoomConcurrencyTest`, `RoomManagerConcurrencyTest`, `SessionManagerConcurrencyTest`, and `UserManagerConcurrencyTest`. These verify safe concurrent operations, serialization of writes, and isolated session expiration.
- **Integration Tests:** Tests such as `ServerConcurrencyTest` and `LogoutTest` exercise the complete client-server pipeline, verifying message broadcasting, session lifecycles, and token invalidation.

10 AI Usage

The group leveraged AI tools to assist with tasks such as building Gradle files, brainstorming, debugging, making minor tweaks, and writing code documentation. AI was strictly excluded from system design decisions, thread manipulation, and other core objectives. Additionally, AI was not used to compose this report beyond basic grammar checking.

11 Conclusion

This project produced a working, multi-user chat server that handles concurrent connections, room management, message broadcasting, and session recovery. The main lessons from the implementation echo what the first assignment showed at the hardware level: getting the *structure* right matters more than micro-optimising individual pieces.

Using a thread-per-client model with per-client outgoing queues kept the concurrency model simple and auditable. The explicit state machine eliminated a whole category of protocol bugs that appeared in earlier prototypes. Implementing the BBQ from scratch, while slower than using a standard library class, gave us a clearer understanding of the blocking and signalling semantics that all concurrent queues rely on. Storing room history synchronously to disk, despite being the less “clever” option, gave us durability essentially for free with no extra code.

The AI room integration was a worthwhile addition: it required only a thin HTTP client wrapping the Ollama API and slotted naturally into the existing room architecture without any special-casing in the handler or state machine.

If we were to extend this system, the most valuable improvements would be proper token expiry and revocation, replacing the file-based user database with a real database, and adding a more sophisticated reconnection strategy with exponential backoff.

12 GitLab Repository and Work Distribution

The entirety of the project (source code, build scripts, and documentation) can be found on GitLab: [Repository](#)

A README was included to allow for an easier understanding of the repository structure, to note prerequisites, and to explain compilation and execution.

Regarding work distribution, there are no discrepancies to note; every member contributed actively and is well aware of the overall design and implementation.